

AWS Cloud Project

Project1

AWS Cloud Infrastructure

Dipesh Poudel

Cloud Enthusiast

VPC

EC2

ALB

ASG

RDS

S3

CloudFront

CloudWatch

IAM

ACM

Secrets Manager

Route 53

17

AWS Services

6

Subnets

2

Avail. Zones

< \$1

Total Cost

June 2026 | ap-south-1 (Mumbai) | Portfolio Project

1. Project Overview

Designed and deployed a production-grade, scalable web application infrastructure on Amazon Web Services (AWS) from scratch. The project simulates a real-world cloud environment — covering networking, compute, security, database, content delivery, and monitoring across multiple Availability Zones.

Attribute	Details
Project Name	Project1 — AWS Cloud Infrastructure
Built By	Dipesh Poudel
Region	ap-south-1 (Mumbai)
Live URL	dipeshpoudel.info.np
Build Time	~6 hours
Total Cost	< \$0.50 USD
Status	Successfully deployed, tested, and decommissioned

2. Architecture Diagram

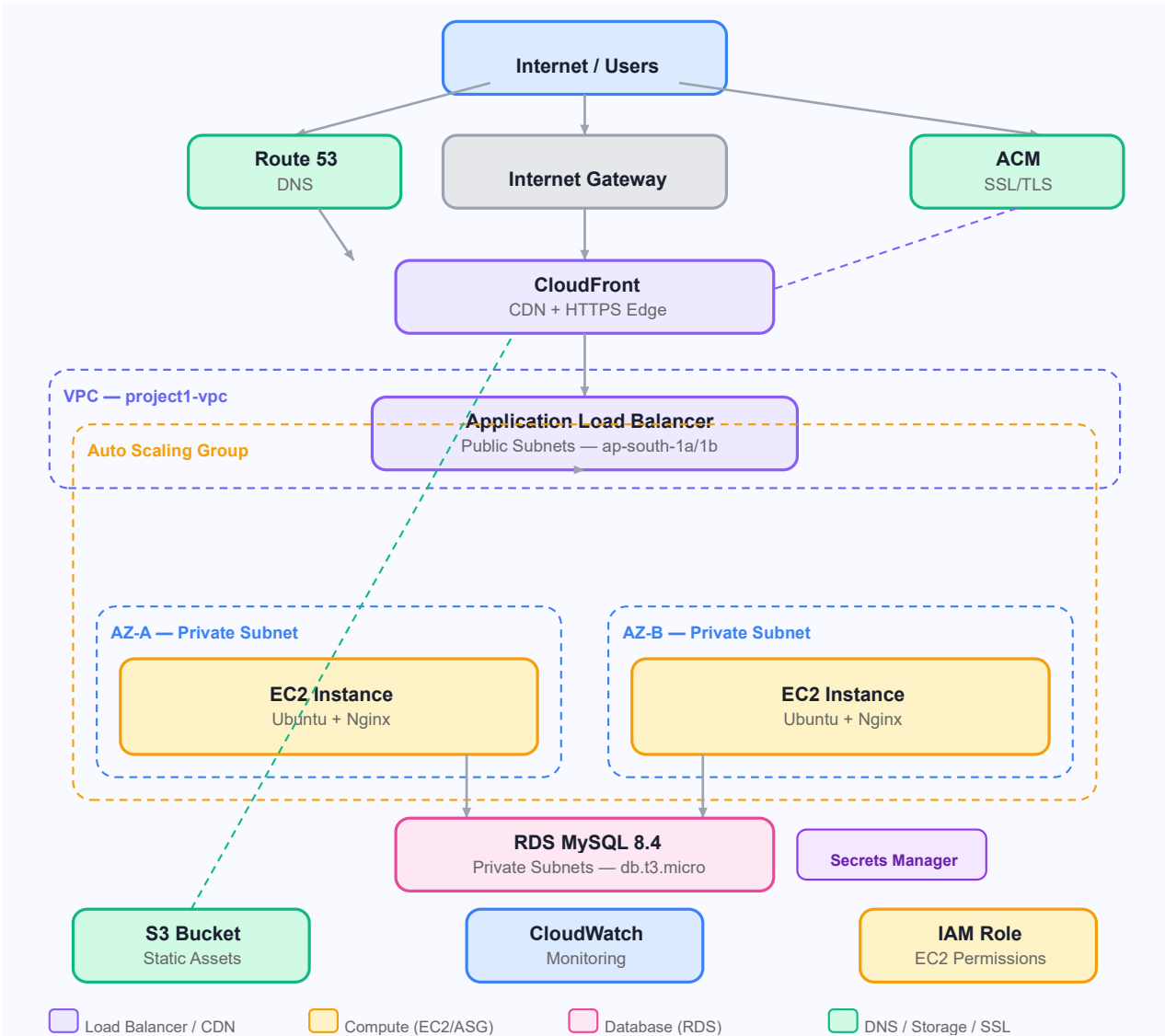


Figure 1 Project1 AWS Structure

3. AWS Services Used

App Load Balancer	HTTPS traffic distribution across Availability Zones	Compute
Target Groups	Instance registration with /health.html health checks	Compute
Amazon RDS MySQL	Managed relational database in private subnet	Database
Secrets Manager	Secure DB credential storage — no hardcoded passwords	Security
Amazon S3	Static asset hosting with versioning enabled	Storage
Amazon CloudFront	Global CDN with HTTPS, OAC, and custom domain	CDN
AWS Certificate Mgr	SSL/TLS certificate via DNS validation (us-east-1)	Security
Amazon CloudWatch	Monitoring dashboard, alarms, and SNS notifications	Monitoring
AWS IAM	EC2 role with least-privilege permissions — no access keys	Security

Service	Purpose	Tier
Amazon VPC	Custom network isolation with public/private subnets	Networking
Internet Gateway	Public internet access for ALB	Networking
NAT Gateway	Outbound internet for private EC2 instances	Networking
Route Tables	Traffic routing between subnets and gateways	Networking
Security Groups	Tier-to-tier traffic control (ALB → EC2 → RDS)	Security
Amazon EC2	Ubuntu 24.04 web server running Nginx	Compute
Launch Template	Reusable, versioned EC2 configuration with user data	Compute
Auto Scaling Group	Automated instance replacement and CPU-based scaling	Compute

4. Network Design

VPC CIDR: 10.0.0.0/16 — 6 subnets across 2 Availability Zones

Subnet Name	CIDR	AZ	Type	Purpose
project1-public-1a	10.0.1.0/24	ap-south-1a	Public	ALB
project1-public-1b	10.0.2.0/24	ap-south-1b	Public	ALB
project1-private-app-1a	10.0.3.0/24	ap-south-1a	Private	EC2
project1-private-app-1b	10.0.4.0/24	ap-south-1b	Private	EC2
project1-private-db-1a	10.0.5.0/24	ap-south-1a	Private	RDS
project1-private-db-1b	10.0.6.0/24	ap-south-1b	Private	RDS

Key Design Decision: ALB sits in public subnets and faces the internet. EC2 and RDS have no public IPs — accessible only through their upstream services. This is the standard production security pattern and the exact fix applied over the previous TechNova project where everything was in public subnets.

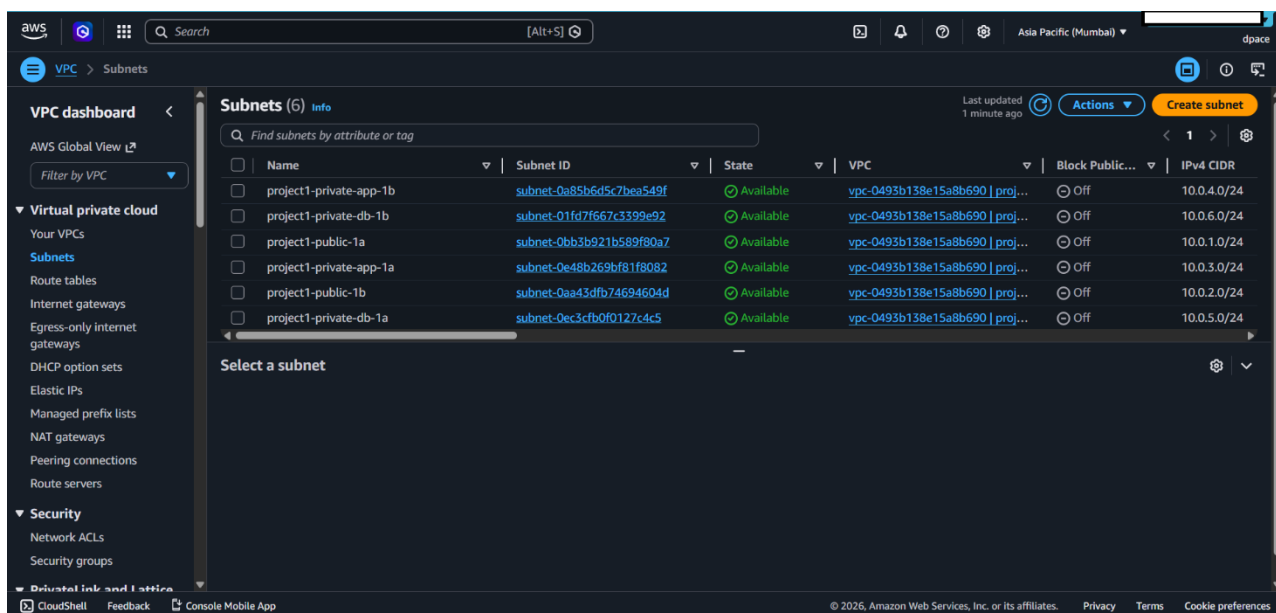


Figure 2 Subnets

5. Security Group Architecture

A strict 3-tier security group chain ensures no traffic skips a layer. This is the direct fix for the health check failures encountered in the previous TechNova project.

Security Group	Inbound Port	Source	Reason
project1-alb-sg	HTTP 80 / HTTPS 443	0.0.0.0/0 (internet)	Public web traffic
project1-ec2-sg	HTTP 80	project1-alb-sg ONLY	ALB to EC2 only
project1-ec2-sg	SSH 22	My IP only	Admin access

project1-rds-sg

MySQL 3306

project1-ec2-sg ONLY

EC2 to DB only

Root cause of TechNova failure: EC2 Security Group had port 80 open to 0.0.0.0/0 instead of project1-alb-sg. This caused intermittent ALB health check failures, triggering endless Auto Scaling replacement loops. One rule change fixed everything.

6. Implementation Phases

Phase 1 — Network Foundation	Built a custom VPC with 6 subnets across 2 Availability Zones. Configured Internet Gateway for public access and NAT Gateway for private subnet outbound connectivity. Created separate route tables for public (IGW) and private (NAT) tiers. Enabled auto-assign public IP on public subnets only.
Phase 2 — Security Groups	Implemented strict tier-to-tier Security Group rules. EC2 accepts traffic only from ALB. RDS accepts traffic only from EC2. SSH restricted to admin IP only. Nothing exposed to internet except the ALB.
Phase 3 — Compute + Load Balancing + Auto Scaling	Created Launch Template with Ubuntu 24.04, Nginx, and user data script. Configured Target Group with explicit /health.html health check, 120s grace period, and ELB health check type. Deployed ALB in public subnets with ASG in private subnets. CPU target tracking at 60%.
Phase 4 — Database + Secrets Management	Launched RDS MySQL 8.4 (db.t3.micro) in isolated private DB subnets with no public access. Created DB subnet group across both AZs. Stored all credentials in AWS Secrets Manager — no hardcoded passwords. EC2 accesses secrets via IAM role, not access keys.
Phase 5 — CDN + HTTPS + Domain	Configured S3 bucket for static assets with CloudFront Origin Access Control (OAC) — S3 is never publicly accessible. Provisioned ACM certificate in us-east-1 via DNS validation through Cloudflare. Configured CloudFront with HTTPS redirect and custom domain dipeshpoudel.info.np.
Phase 6 — Monitoring	Set up CloudWatch dashboard with 4 real-time widgets: CPU utilization, healthy host count, request count, and RDS free storage. Created 3 alarms with SNS email notifications: unhealthy hosts below 1, CPU above 70%, RDS storage below 500MB.

7. Key Challenges and Resolutions

Challenge 1 — Auto Scaling Health Check Loop

Problem	In the previous TechNova project, instances were continuously replaced by Auto Scaling. Root cause was never identified at the time.
Root Cause	EC2 Security Group port 80 was open to 0.0.0.0/0 instead of project1-alb-sg. ALB health check traffic was not explicitly trusted, causing intermittent failures.

Fix Applied	Changed EC2-SG inbound rule source from 0.0.0.0/0 to project1-alb-sg. Changed health check path to /health.html. Set grace period to 120 seconds. Changed health check type to ELB.
Result	Target Group immediately showed Healthy. Zero replacement loops.

Challenge 2 — RDS Instance Type Restriction

Problem	AWS console Sandbox template did not allow selecting db.t3.micro in ap-south-1.
Root Cause	Sandbox template restricts certain instance types in specific regions.
Fix Applied	Switched to Dev/Test template with Single-AZ deployment.
Result	db.t3.micro available immediately. Same cost, no restrictions.

Challenge 3 — ACM Certificate Region

Problem	Created SSL certificate in ap-south-1 — CloudFront rejected it.
Root Cause	CloudFront only accepts ACM certificates from us-east-1 regardless of infrastructure region.
Fix Applied	Created new certificate in us-east-1. Existing Cloudflare CNAME records validated the new cert instantly.
Result	Certificate issued in under 2 minutes. CloudFront distribution deployed with HTTPS successfully.

8. Testing Performed

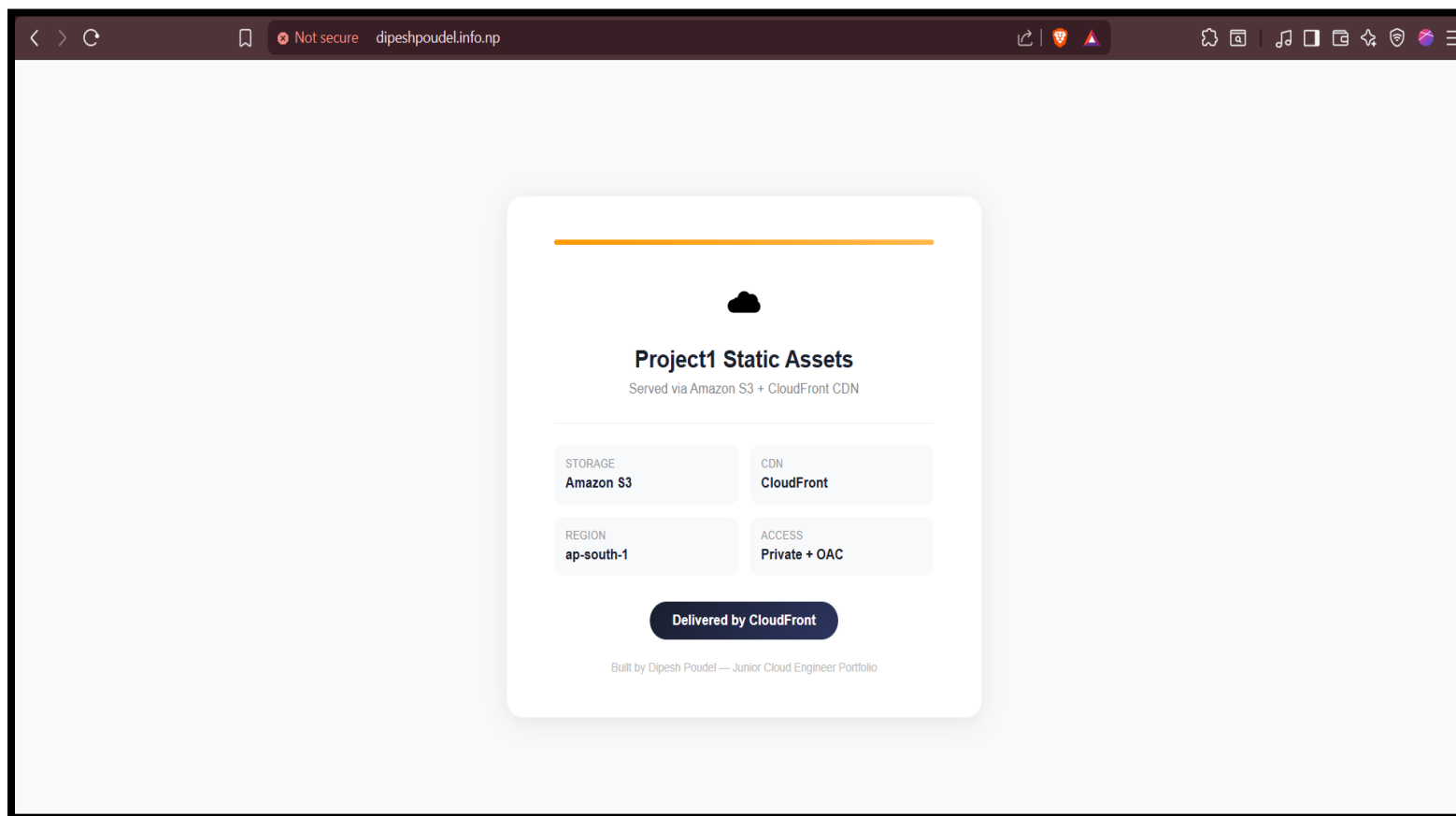


Figure 3 Hosted Domain Test

Test	Action	Result
Test 1 — ASG Self Healing	Manually terminated the EC2 instance via AWS Console.	ASG detected unhealthy instance, launched replacement in private subnet, registered with Target Group. Site recovered automatically within 3-4 minutes.
Test 2 — Health Check Failure Simulation	Removed port 80 inbound rule from EC2 Security Group.	Target Group immediately marked instance unhealthy. ALB returned 503. Re-added rule with ALB-SG source — health checks recovered within 60 seconds.
Test 3 — CloudWatch Dashboard Verification	Monitored dashboard during build and testing phases.	CPU spike visible during instance launch. HealthyHostCount dropped to 0 during termination test then recovered. RequestCount showed traffic patterns correctly.

9. Key Learnings

Networking

- Public/private subnet separation and why it matters for security
- NAT Gateway role for private instance outbound connectivity
- Route table design for multi-tier VPC architectures

Security

- Tier-to-tier Security Group chaining as the production standard
- Why EC2 should never have a public IP in properly designed architecture
- IAM roles over access keys for service-to-service authentication
- Secrets Manager as the production standard for credential management

Compute & Scaling

- How ELB health checks differ from EC2 health checks in ASG
- Why health check grace period matters during instance startup
- Launch Templates enable consistent, repeatable infrastructure
- Target tracking scaling policies for automated capacity management

CDN & SSL

- ACM certificates for CloudFront must be in us-east-1 regardless of origin region
- CloudFront Origin Access Control (OAC) keeps S3 buckets private
- DNS validation for SSL certificate provisioning through third-party DNS

Monitoring

- CloudWatch alarms for proactive infrastructure monitoring
- SNS topics for alert routing to email notification
- Dashboard design for operational visibility across services

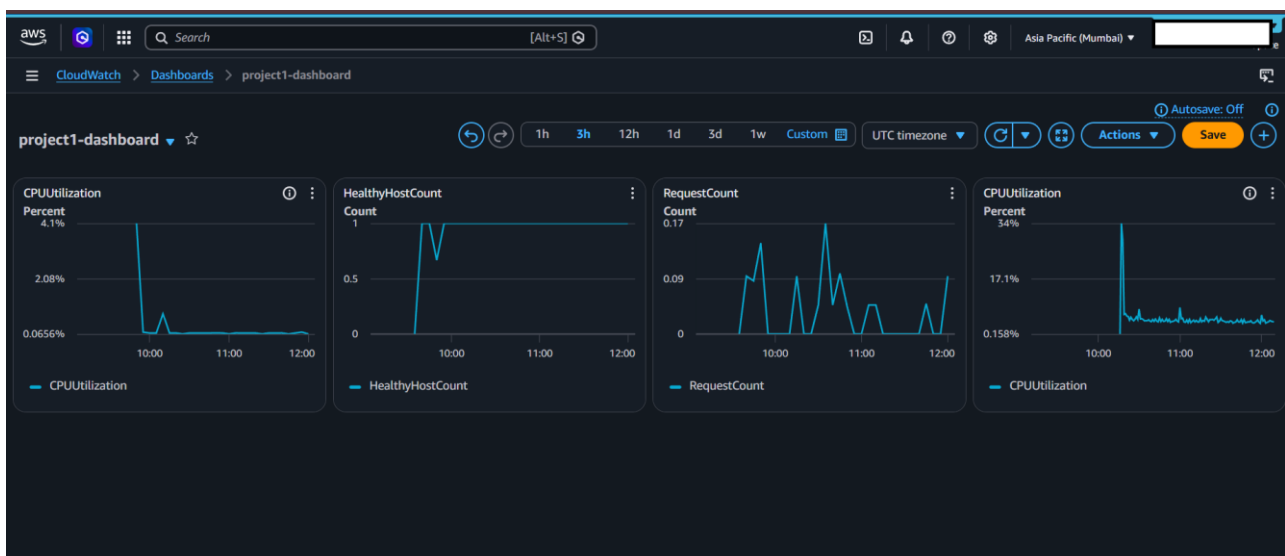


Figure 4 Cloudwatch Project1 Dashboard

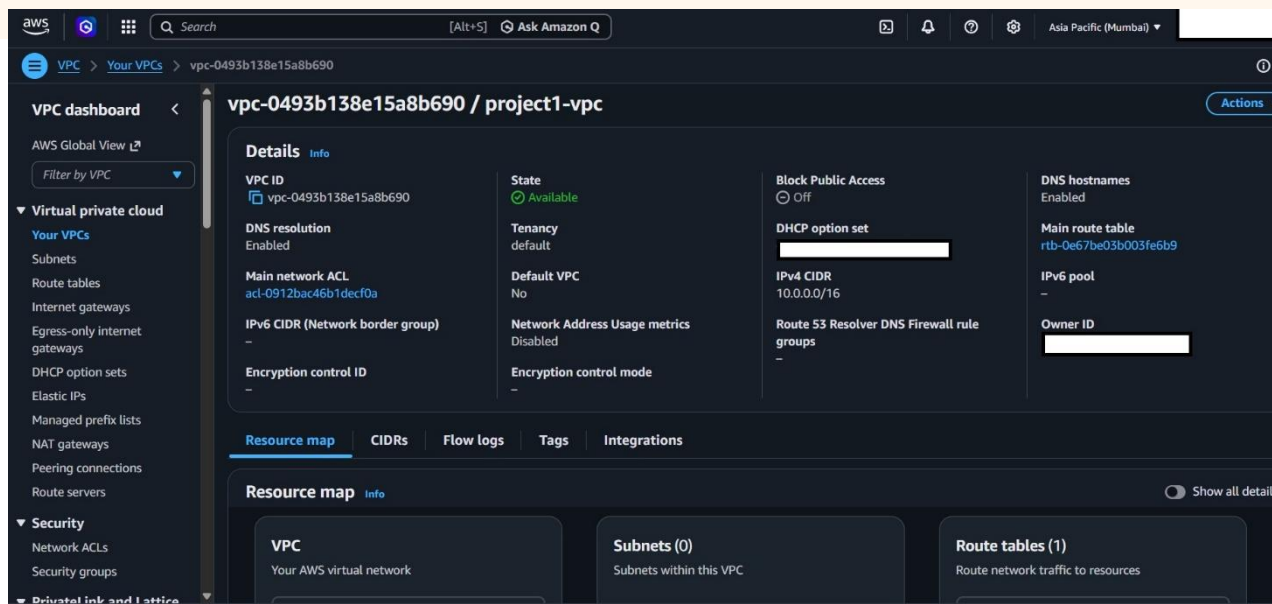


Figure 5 Project1 VPC

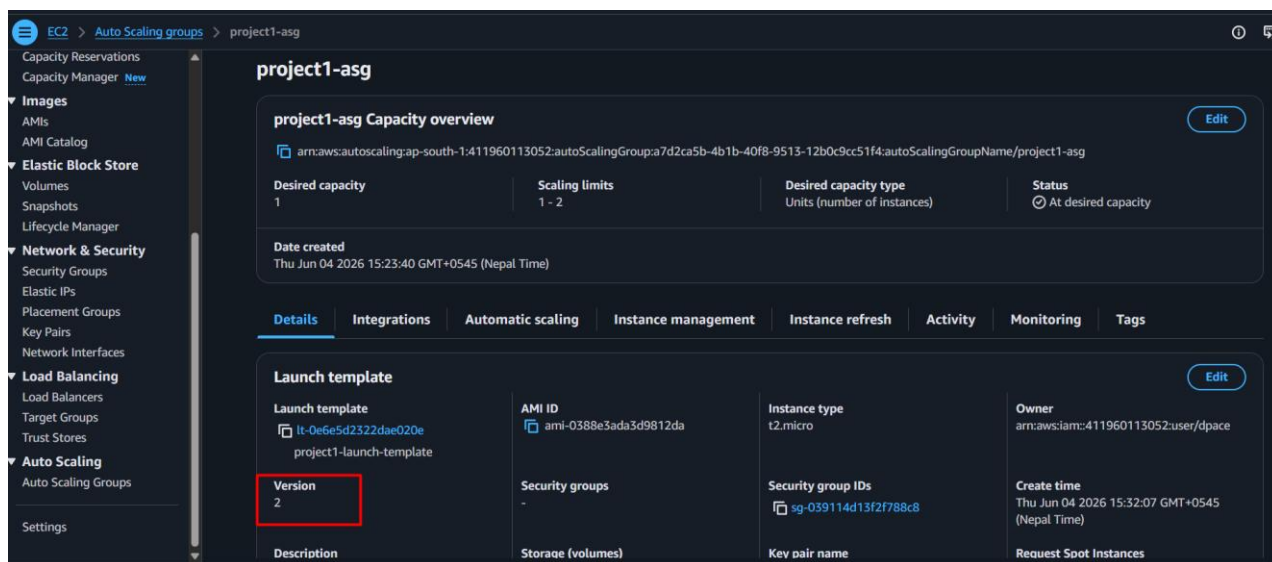


Figure 6 Launch Template Project1 Version 2

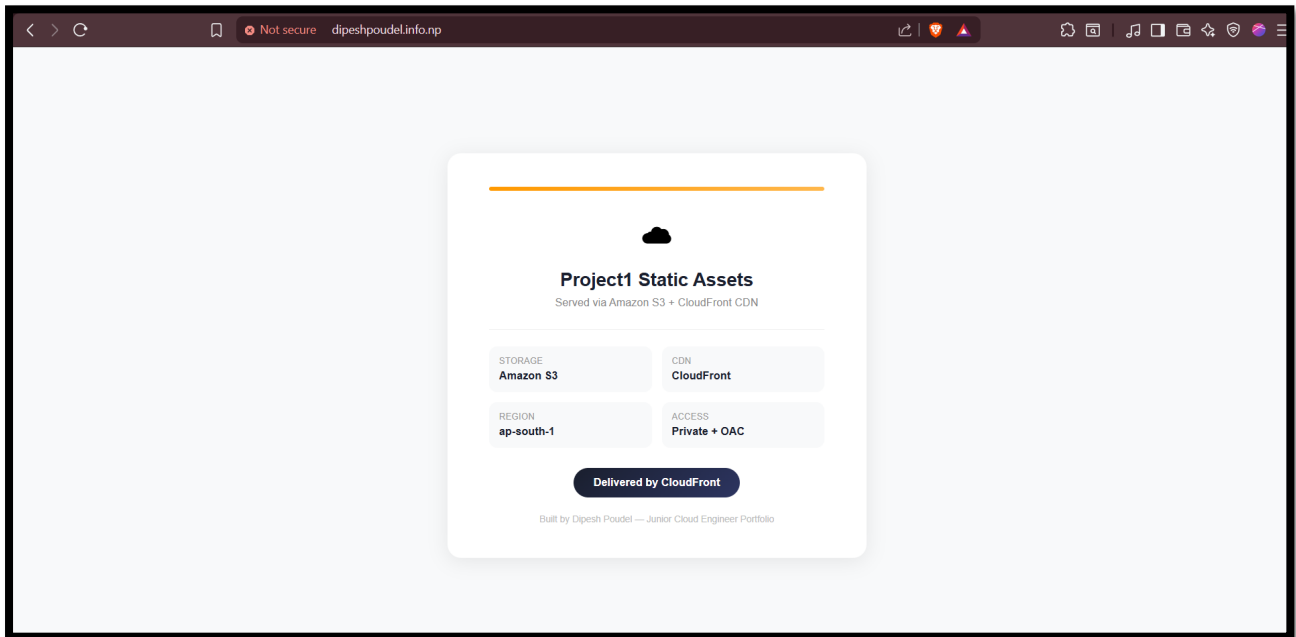


Figure 7 Hosted in Domain with AM and CDN

